

Matthew So
 Dr. Samit Shivadekar
 CMSC 471
 23 September 2026

Project Assignment 1

Question 1

Steps	Scrabble	Frequency
$h(n)$ = the number of indexes where n differs from the goal	$h(n) = \sum_{i=1}^{len(word)} (c(g_i), \text{ if } n_i \neq g_i)$ Where c is Scrabble cost based on g_i, and g_i is the letter at position i in the goal word	$h(n)$ = the number of indexes where n differs from the goal
Each mismatched position must be changed at least once to reach the goal. Since every change costs exactly 1 using steps cost, the number of mismatched positions represents an unavoidable amount of remaining cost.	For every position where the current word differs from the goal, the goal letter must eventually be inserted into that position. Thus, the Scrabble value of each of those goal letters is an unavoidable part of the remaining cost.	This heuristic is the same as the steps heuristic. Every mismatched position must be changed at least once to reach the goal. If there are n remaining mismatched positions, it will have at least n remaining cost.
Each mismatched position requires at least one action, so if there are n mismatched positions, every solution must cost at least n steps. Our heuristic gives n .	Every mismatched position must eventually contain its corresponding goal letter. Therefore, the Scrabble cost of inserting each of those goal letters must be paid at least once. The heuristic only counts these unavoidable costs.	My heuristic doesn't include any rarity costs, so it cannot accidentally overestimate the actual cost that includes rarity calculations.

Question 2

Word Pair	Cost Measure	UCS cost	UCS expanded	A* cost	A* expanded	Reduction
<i>WHY</i> → <i>ASK</i>	Steps	5	773	5	16	97.93%
<i>WHY</i> → <i>ASK</i>	Scrabble	9	765	9	13	98.30%
<i>WHY</i> → <i>ASK</i>	Frequency	38.86	808	38.86	755	6.56%
<i>BEAR</i> → <i>DUCK</i>	Steps	4	691	4	6	99.13%
<i>BEAR</i> → <i>DUCK</i>	Scrabble	11	2981	11	6	99.80%
<i>BEAR</i> → <i>DUCK</i>	Frequency	38.24	1084	38.24	720	33.58%

1. Yes, UCS and A* always returned the same optimal cost.
2. The largest percentage reduction per word pair is **Scrabble**. *WHY* → *ASK* has a reduction of 98.30% and *BEAR* → *DUCK* has 99.80%.
3. The least informative heuristic per word pair is **Frequency**. *WHY* → *ASK* has a reduction of 6.56% and *BEAR* → *DUCK* has 33.58%.
4. Scrabble's heuristic tracks the true remaining cost tightly compared to Frequency's heuristic. Scrabble computes the exact substitution cost for each mismatched position from the same table inside `path_cost`. I was able to use `path_cost` on Scrabble's heuristic calculation because Scrabble depended only on the letter being inserted, and not the resulting word or path history. On the other hand, Frequency is significantly worse because its true cost, $1 + \text{rarity}(w)$, depends on the specific word actually entered, and that word isn't known until a path is found. My heuristic instead adds 1 per mismatched letter, which assumes each future word has a rarity of 0. For a better heuristic, it should include rarity within the calculation.

Question 3

- Steps: Yes, the optimal cost from $W_1 \rightarrow W_2$ is the same as $W_2 \rightarrow W_1$

Every action costs 1, thus cost only depends on the number of actions. Reversing the path gives the same length, just in the other direction. $W_1 \rightarrow W_2$ and $W_2 \rightarrow W_1$ has the same number of actions, meaning the same cost.

- Scrabble: No

Cost depends on the letter being inserted, not removed. When you reverse a path, it swaps which letters get inserted. Differing letters being inserted means differing costs.

Counterexample:

BEAR → *BEAK* → *BECK* → *BUCK* → *DUCK*: $k = 5, c = 3, u = 1, d = 1$, cost of 11

DUCK → *BUCK* → *BECK* → *BEAK* → *BEAR*: $b = 3, e = 1, a = 1, r = 1$, cost of 6

- Frequency: No

Frequency charges the rarity of the word you end on, not the word you start from. Thus, going one direction uses the goal's rarity and going the other uses the start's rarity, and those likely differ.

Counterexample:

BEAR → *BEAK* → *BECK* → *BUCK* → *DUCK*: with a cost of 38.24

DUCK → *BUCK* → *BECK* → *BEAK* → *BEAR*: with a cost of 36.79

Question 4

1. GPT-5.6 Luna, prompted the exact image:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Its proposed heuristic: Instead of just counting how many letters are wrong, also estimate how expensive it will be to fix each wrong letter.

2. Let the current state be $n = n_1 \dots n_k$, the goal be $g = g_1 \dots g_k$ and let $R(w)$ denote the dictionary rarity of the word w

For each position i , precompute $m_i = \min\{R(w) : w \text{ is in the dictionary and } w_i = g_i\}$

Then define $\sum_{i:n_i \neq g_i} (1 + m_i)$ with $h(g) = 0$

3. It is admissible because for every position i for which $n_i \neq g_i$, that position must at some point change from a non goal letter to g_i . Let w be the word entered by that particular move. The move costs $1 + R(w)$. Since $w_i = g_i$, then $R(w) \geq m_i$. Therefore, the move costs at least $1 + m_i$.

One legal action changes only one position, so the moves responsible for fixing the different initial incorrect positions are distinct. Thus, every solution has remaining cost of at least

$$\sum_{i:n_i \neq g_i} (1 + m_i) = h(n)$$

Thus, h never overestimates the optimal remaining cost.

4. Tests done on *WHY* → *ASK*, *BEAR* → *DUCK*, *CAT* → *DOG*, *DOG* → *CAT*:

	Nodes expanded, Original Frequency Heuristic	Nodes expanded, ProposedFrequency Heuristic
<i>WHY</i> → <i>ASK</i>	755	510
<i>BEAR</i> → <i>DUCK</i>	720	319
<i>CAT</i> → <i>DOG</i>	211	85
<i>DOG</i> → <i>CAT</i>	129	80

All solution costs are the same between the two heuristics. This shows that the proposed heuristic consistently reduces the number of expanded nodes.